CLOUD

CHEAT SHEETS

The ideas every cloud shares, side by side on AWS, Azure and Google Cloud

IaaS / PaaS / SaaS · Regions & Zones · Accounts & Projects · Pricing · IAM · Workload Identity · Secrets & KMS · Guardrails · VPC / VNet · Firewalls · Private Endpoints · Load Balancing · DNS · CDN · VMs · Containers · Kubernetes · Functions · Object Storage · Databases · Queues & Events · Terraform · OpenTelemetry · CI/CD OIDC · Well-Architected · FinOps

26 REFERENCE SHEETS

CONTENTS

CLOUD BASICS

01 What the Cloud Is
02 Regions, Zones & the Edge
03 Accounts, Projects & Hierarchy
04 Shared Responsibility & Pricing Models

IDENTITY & SECURITY

05 Identity & Access Concepts
06 Workload Identity & Federation
07 Secrets & Encryption
08 Security Baseline & Guardrails

NETWORKING

09 Virtual Networks & Subnets
10 Firewalls & Private Access
11 Load Balancing & DNS
12 CDN & Edge

COMPUTE

13 Virtual Machines & Autoscaling
14 Containers & Managed Kubernetes
15 Serverless Functions
16 Choosing Compute

STORAGE, DATA & MESSAGING

17 Object, Block & File Storage
18 Managed Databases
19 Messaging & Events

OPERATIONS

20 Infrastructure as Code
21 Observability
22 CI/CD to the Cloud

ARCHITECTURE & COST

23 Reliability & Well-Architected
24 Cost Management & FinOps

QUICK REFERENCE

25 Quick Reference — aws · az · gcloud
26 Quick Reference — Service Map

Covers AWS, Azure and Google Cloud side by side · aws / az / gcloud · Terraform · links into each provider set (2026)

Begin → Sheet 01

CLOUD

Table of Contents

26 sheets · syntax · examples · notes

CLOUD BASICS 01 – 04

- 01 What the Cloud Is
- 02 Regions, Zones & the E…
- 03 Accounts, Projects & H…
- 04 Shared Responsibility …

IDENTITY & SECURITY 05 – 08

- 05 Identity & Access Con…
- 06 Workload Identity & Fe…
- 07 Secrets & Encryption
- 08 Security Baseline & Gu…

NETWORKING 09 – 12

- 09 Virtual Networks & Su…
- 10 Firewalls & Private Acc…
- 11 Load Balancing & DNS
- 12 CDN & Edge

COMPUTE 13 – 16

- 13 Virtual Machines & Aut…
- 14 Containers & Managed…
- 15 Serverless Functions
- 16 Choosing Compute

STORAGE, DATA & MESSAGING 17 – 19

- 17 Object, Block & File St…
- 18 Managed Databases
- 19 Messaging & Events

OPERATIONS 20 – 22

- 20 Infrastructure as Code
- 21 Observability
- 22 CI/CD to the Cloud

ARCHITECTURE & COST 23 – 24

- 23 Reliability & Well-Archi…
- 24 Cost Management & Fi…

QUICK REFERENCE 25 – 26

- 25 Quick Reference — aw…
- 26 Quick Reference — Ser…

Sheet 01

What the Cloud Is

on-demand self-service · IaaS / PaaS / SaaS / FaaS · managed vs self-managed · elasticity · pay-as-you-go · the big three and the rest

RENTING COMPUTING INSTEAD OF OWNING IT

A cloud provider runs data centers full of servers, storage and networks and rents them out through an API. You ask for a virtual machine, a database or a storage bucket and get it in minutes, pay only while you use it, and hand back what you no longer need. **AWS**, **Microsoft Azure** and **Google Cloud** are the three largest providers; most of their ideas match, only the names differ.

KEY POINTS

- Everything is an API call: the web console, CLI, SDKs and IaC tools all call the same APIs
- The more the provider manages, the less you control (and the less you maintain)
- Elasticity: scale up for a spike, back down afterwards
- This set teaches each idea once; the provider sets show the details

THE SAME REQUEST ON THREE CLOUDS

```
# Who am I, and which account / subscription / project am I in?
aws sts get-caller-identity
az account show --output table
gcloud config list

# Create a storage bucket (object storage, sheet 17)
aws s3 mb s3://demo-bucket-7f3a --region us-east-1
az storage account create -n demostore7f3a -g rg-demo -l eastus2
az storage container create -n demo --account-name demostore7f3a
gcloud storage buckets create gs://demo-bucket-7f3a --location=us-central1

# Delete it when done (you pay while it exists)
aws s3 rb s3://demo-bucket-7f3a
az storage account delete -n demostore7f3a -g rg-demo --yes
gcloud storage buckets delete gs://demo-bucket-7f3a
```

CODE EXPLAINED

- → Each provider has one main CLI: `aws`, `az` and `gcloud`
- → Bucket names must be globally unique, hence the random suffix
- → Azure puts containers inside a **storage account**, so it needs two commands
- → Deleting what you made is part of the workflow: idle resources still bill

ACROSS THE CLOUDS

- ↔ **AWS** the account and region model (aws-01)
- ↔ **Azure** tenants, subscriptions and resource groups (az-01)
- ↔ **Google Cloud** organizations and projects (gcp-01)

RULE OF THUMB

- ☞ Use managed services (PaaS) unless you have a reason to run the server yourself
- ☞ Pick one provider to learn deeply; the concepts transfer
- ☞ Script every change: a click you cannot repeat is a click you will repeat wrong

SERVICE MODELS

MODEL	YOU MANAGE	EXAMPLES
IaaS	OS, runtime, app, data	EC2, Azure VMs, Compute Engine
PaaS	App and data	Elastic Beanstalk, App Service, App Engine
Serverless / FaaS	Code only	Lambda, Azure Functions, Cloud Run
SaaS	Your settings	Microsoft 365, Gmail, Salesforce

FIVE TRAITS OF CLOUD (NIST)

TRAIT	MEANS
On-demand self-service	No ticket to IT
Broad network access	Reach it over the internet
Resource pooling	Shared hardware, isolated tenants
Rapid elasticity	Scale in minutes
Measured service	Pay for what you use

DO & DON'T

- ✓ **Do** Tag every resource with an owner and purpose from day one
- ✓ **Do** Set a budget alert before you create anything
- ✗ **Don't** Lift and shift everything onto VMs and call it cloud-native
- ✗ **Don't** Leave test resources running overnight "just in case"

LOCAL TESTING

- ⌂ Practice in a free-tier account with a budget alert set to a few dollars
- ⌂ Use the browser shells (CloudShell, Azure Cloud Shell, Cloud Shell): CLIs are preinstalled and authenticated
- ⌂ Emulators for single services appear on later sheets (LocalStack, Azurite, Google emulators)

PRODUCTION CHECKLIST

- ☐ Separate accounts / subscriptions / projects for dev, test and prod
- ☐ Billing alerts and an owner tag on every resource
- ☐ Everything created by IaC, not by hand (sheet 20)

Sheet 02

Regions, Zones & the Edge

regions · availability zones · edge / points of presence · latency & data residency · zonal vs regional vs global resources · choosing a region

WHERE YOUR RESOURCES LIVE

Each provider splits the world into **regions** (a metro area such as Northern Virginia or Iowa), and each region into **availability zones**: separate data centers with their own power and cooling, close enough for fast links. Spreading an app across zones survives a data-center failure; spreading across regions survives a regional outage. **Edge locations** cache content and terminate connections close to users.

KEY POINTS

- Region names differ: `us-east-1` (AWS), `eastus2` (Azure), `us-central1` (Google Cloud)
- Zones: `us-east-1a`, Azure zones `1` / `2` / `3`, `us-central1-a`
- Not every service or instance type exists in every region
- Data moving between regions is billed; within a zone it is usually free

LISTING REGIONS AND ZONES

```
# AWS: regions enabled for the account, then zones in one region
aws ec2 describe-regions --query "Regions[].RegionName" --output text
aws ec2 describe-availability-zones --region us-east-1 \
    --query "AvailabilityZones[].ZoneName"

# Azure: locations (regions) and which have availability zones
az account list-locations --query "[].name" -o tsv
az vm list-skus -l eastus2 --zone --size Standard_D2s_v5 -o table

# Google Cloud: regions and the zones inside one
gcloud compute regions list --format="value(name)"
gcloud compute zones list --filter="region:us-central1"

# Set a default so you stop typing it
aws configure set region us-east-1
az config set defaults.location=eastus2
gcloud config set compute/region us-central1
```

CODE EXPLAINED

- → `--query` (AWS, Azure) and `--format` / `--filter` (gcloud) trim output
- → Azure lists zone support per VM size, because zones vary by size
- → Defaults keep commands short; scripts should still pass the region explicitly
- → Each provider has 30+ regions; pick close to users and data rules

ACROSS THE CLOUDS

- ↔ **AWS** regions, AZs, Local Zones, edge locations (aws-01)
- ↔ **Azure** regions, availability zones, region pairs (az-01)
- ↔ **Google Cloud** regions, zones, global network tiers (gcp-01)

RULE OF THUMB

- ☞ Pick the region closest to most users that offers every service you need
- ☞ Run production across at least two zones
- ☞ Go multi-region only when the business needs it: it doubles cost and complexity

RESOURCE SCOPE

SCOPE	EXAMPLES
Zonal	A VM, a disk
Regional	A subnet (AWS, Azure), a managed database with zone replicas, a regional bucket
Global	IAM, DNS, CDN, Google Cloud VPCs
Edge	CDN caches, global load balancer front ends

CHOOSING A REGION

FACTOR	QUESTION
Latency	Where are the users?
Data residency	Must data stay in a country?
Services	Is every service available there?
Price	Prices vary by region (10–30%)
Carbon	Some regions run on cleaner energy

DO & DON'T

- ✓ **Do** Put the region in resource names or tags so you can find strays
- ✓ **Do** Check service availability per region before designing
- ✗ **Don't** Assume zone `a` in your account is the same building as zone `a` in another (AWS maps names per account)
- ✗ **Don't** Send traffic between regions without budgeting for it

LOCAL TESTING

- ⌂ Region and zone choices cannot be emulated; test failover in a cheap dev deployment
- ⌂ Use the providers' latency test pages to compare regions from your location
- ⌂ Chaos tools (AWS Fault Injection Service, Azure Chaos Studio) simulate zone loss

PRODUCTION CHECKLIST

- ☐ Spread instances and databases across 2–3 zones
- ☐ Document the backup region and how to fail over to it
- ☐ Keep data in regions allowed by your compliance rules

Accounts, Projects & Hierarchy

AWS Organizations & accounts · Azure tenants, management groups, subscriptions & resource groups · Google Cloud organization, folders & projects · billing boundaries · blast radius

HOW RESOURCES ARE GROUPED

Every cloud has a container that owns resources, bills them and limits who can touch them: an **AWS account**, an **Azure subscription** (with **resource groups** inside) or a **Google Cloud project**. Above that sits an organization tree for policy and billing. Separate containers per environment and team keep a mistake in dev from reaching production.

KEY POINTS

- AWS: Organization → OUs → accounts; the account is the security boundary
- Azure: Entra tenant → management groups → subscriptions → resource groups
- Google Cloud: organization → folders → projects
- Policies set high in the tree are inherited below

CREATING THE CONTAINER

```
# AWS: a new member account in your organization
aws organizations create-account \
    --email dev-team@example.com --account-name "app-dev"
aws organizations list-accounts --query "Accounts[].[Name,Id]" \
    --output table

# Azure: pick the subscription, then make a resource group in it
az account list -o table
az account set --subscription "app-dev"
az group create --name rg-app-dev --location eastus2 \
    --tags owner=team-a env=dev

# Google Cloud: a project under a folder, linked to billing
gcloud projects create app-dev-4821 --folder=123456789012
gcloud billing projects link app-dev-4821 \
    --billing-account=0X0X0X-0X0X0X-0X0X0X
gcloud config set project app-dev-4821
```

CODE EXPLAINED

- → AWS accounts are created from the management account of an Organization
- → An Azure resource group is a folder for resources with a shared lifecycle
- → Google Cloud project IDs are global and permanent; pick them carefully
- → All three need a billing link before resources can be created

ACROSS THE CLOUDS

- ↔ **AWS** Organizations, OUs, accounts (aws-02)
- ↔ **Azure** management groups, subscriptions, resource groups (az-01, az-02)
- ↔ **Google Cloud** organization, folders, projects, billing accounts (gcp-01, gcp-02)

RULE OF THUMB

- ☞ One account / subscription / project per environment per app is a good default
- ☞ Delete a whole resource group or project to clean up everything at once
- ☞ Apply guardrail policies at the top of the tree, not resource by resource

TERMINOLOGY MAP

IDEA	AWS	AZURE	GOOGLE CLOUD
Top level	Organization	Tenant + root mgmt group	Organization
Grouping	OU	Management group	Folder
Billing + isolation	Account	Subscription	Project
Lifecycle group	Tags / stacks	Resource group	Project / labels
Guardrails	SCPs	Azure Policy	Organization Policy

DO & DON'T

- ✓ **Do** Keep production in its own account / subscription / project
- ✓ **Do** Use a naming scheme: app-env-region (rg-shop-prod-eus2)
- ✗ **Don't** Run everything in one shared account where any mistake hits all
- ✗ **Don't** Use the AWS root user or a Global Administrator for daily work

LOCAL TESTING

- ⌂ Create a sandbox account / subscription / project you can delete whole
- ⌂ Google Cloud: `gcloud projects delete` cleans up everything in a test project
- ⌂ Azure: `az group delete -n rg-test --yes --no-wait` removes a test environment

PRODUCTION CHECKLIST

- ☐ Organization-level guardrails (SCPs, Azure Policy, Organization Policy) in place
- ☐ Break-glass admin accounts with MFA, stored offline
- ☐ Billing alerts per account / subscription / project

Shared Responsibility & Pricing Models

WHO SECURES WHAT, AND HOW YOU PAY

Under the **shared responsibility model** the provider secures the hardware, facilities and the services it runs; you secure your data, identities, configuration and code. The more managed the service, the more the provider takes on. Pricing follows a few patterns on every cloud: **on-demand**, discounted **commitments**, cheap interruptible **spot** capacity, and **egress** fees for data leaving the cloud.

KEY POINTS

- Misconfiguration (public buckets, open ports, leaked keys) is your responsibility
- Commitments trade flexibility for 30–70% off
- Spot capacity can be reclaimed with short notice (30 s to 2 min)
- Data in is free; data out to the internet is billed per GB

PRICING LOOKUPS AND BUDGETS

```
# AWS: a monthly budget that emails at 80% of $50
aws budgets create-budget --account-id 111122223333 \
    --budget file://budget.json \
    --notifications-with-subscribers file://notify.json

# Azure: budget on the current subscription
az consumption budget create --budget-name monthly-50 \
    --amount 50 --category cost --time-grain Monthly \
    --start-date 2026-12-01 --end-date 2027-12-01

# Google Cloud: budget with thresholds on a billing account
gcloud billing budgets create --billing-account=0X0X0X-0X0X0X-0X0X0X \
    --display-name="monthly-50" --budget-amount=50USD \
    --threshold-rule=percent=0.5 --threshold-rule=percent=0.9

# Spot / preemptible capacity
aws ec2 run-instances --instance-market-options MarketType=spot ...
az vm create ... --priority Spot --eviction-policy Deallocate
gcloud compute instances create ... --provisioning-model=SPOT
```

CODE EXPLAINED

- → Budgets alert; they do not stop spending by themselves
- → AWS budget details live in JSON files passed with `file://`
- → Spot flags differ but mean the same thing: cheap, can be taken back
- → The `...` stands for the usual instance options on each provider

ACROSS THE CLOUDS

- ↔ **AWS** Free Tier, Savings Plans, Spot (aws-02, aws-16)
- ↔ **Azure** free account, Reservations, savings plans, Spot VMs (az-02, az-16)
- ↔ **Google Cloud** Free Tier, committed and sustained use discounts, Spot VMs (gcp-02, gcp-16)

RULE OF THUMB

- ☞ Run steady workloads on commitments, spiky ones on demand, batch on spot
- ☞ Price the network: egress and NAT often surprise more than compute
- ☞ Read the free-tier limits: they are per month and per service

RESPONSIBILITY

LAYER	IAAS	PAAS	SAAS
Data & access	You	You	You
Application	You	You	Provider
OS & runtime	You	Provider	Provider
Network controls	You	Shared	Provider
Hardware & facility	Provider	Provider	Provider

PRICING MODELS

MODEL	DISCOUNT	CATCH
On-demand	None	Highest rate
Commitment (1–3 yr)	~30–70%	Pay even if unused
Spot	Up to ~90%	Can be reclaimed
Free tier	Free up to limits	Limits are small

DO & DON'T

- ✓ **Do** Set budgets with alerts at 50%, 80% and 100%
- ✓ **Do** Use spot for CI runners, batch jobs and stateless workers
- ✗ **Don't** Buy a 3-year commitment for a workload you have run for a month
- ✗ **Don't** Assume "serverless" means free when idle (some plans bill minimums)

LOCAL TESTING

- ⌂ Price designs with the AWS Pricing Calculator, Azure Pricing Calculator and Google Cloud Pricing Calculator
- ⌂ Test spot interruptions with AWS Fault Injection Service, `az vm simulate-eviction`, or by stopping a Google Cloud Spot VM
- ⌂ Infracost estimates Terraform cost changes in pull requests

PRODUCTION CHECKLIST

- ☐ Budget alerts routed to a team channel, not one inbox
- ☐ Cost allocation tags / labels enforced by policy
- ☐ A monthly review of the top 10 cost lines

Sheet 05

Identity & Access Concepts

principals (users, groups, workloads) · roles & permissions · policies & scopes · least privilege · MFA · human vs machine identity

WHO CAN DO WHAT, WHERE

Every request to a cloud API is made by a **principal** (a person, group or workload) and checked against **policies** that grant **permissions** on a **scope** (an account, subscription, project or single resource). AWS writes policies as JSON documents attached to identities or resources; Azure assigns **roles** at a scope with RBAC; Google Cloud binds roles to principals in allow policies.

KEY POINTS

- Deny by default: nothing is allowed until a policy grants it
- An explicit deny beats any allow (AWS, Azure deny assignments, Google deny policies)
- Grant roles to groups, not individual users
- Humans sign in with SSO + MFA; workloads use roles or managed identities (sheet 06)

GRANTING READ ACCESS ON EACH CLOUD

```
# AWS: attach a managed read-only policy to a role
aws iam attach-role-policy --role-name analytics-reader \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Azure: assign a built-in role to a group at resource-group scope
az role assignment create --assignee-object-id <group-object-id> \
    --assignee-principal-type Group \
    --role "Storage Blob Data Reader" \
    --scope /subscriptions/<sub-id>/resourceGroups/rg-data

# Google Cloud: bind a predefined role to a group on a project
gcloud projects add-iam-policy-binding app-prod-4821 \
    --member="group:analysts@example.com" \
    --role="roles/storage.objectViewer"
```

CODE EXPLAINED

- All three express "this principal may read storage on this scope"
- Azure scope is a resource ID path; narrower scope = less access
- Google Cloud role names read like `roles/service.permissionLevel`
- The JSON shows AWS's Effect / Action / Resource structure

AN AWS POLICY DOCUMENT

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::reports", "arn:aws:s3:::reports/*"] }] }
```

ACROSS THE CLOUDS

- **AWS** IAM users, roles, policies (aws-05, aws-06)
- **Azure** Entra ID + Azure RBAC role assignments (az-05, az-06)
- **Google Cloud** IAM principals, roles, allow policies (gcp-05, gcp-06)

RULE OF THUMB

- Start with built-in / predefined roles; write custom ones only for gaps
- Grant at the narrowest scope that works
- Review who has Owner / Administrator monthly

TERMINOLOGY

IDEA	AWS	AZURE	GOOGLE CLOUD
Directory	IAM + Identity Center	Microsoft Entra ID	Cloud Identity / Workspace
Permission bundle	Policy	Role definition	Role
Grant	Attach policy	Role assignment	Role binding
Admin role	AdministratorAccess	Owner	roles/owner
Read-only	ReadOnlyAccess	Reader	roles/viewer

DO & DON'T

- ✓ **Do** Use SSO with MFA for every human
- ✓ **Do** Grant to groups and review group membership
- ✗ **Don't** Hand out Owner / Administrator to unblock someone quickly
- ✗ **Don't** Share one login among a team

LOCAL TESTING

- AWS IAM Policy Simulator tests a policy before you attach it
- Azure: `az role assignment list --assignee <id> --all` shows what someone can do
- Google Cloud Policy Troubleshooter explains why access was allowed or denied

PRODUCTION CHECKLIST

- MFA enforced for every human identity
- No standing admin access; use just-in-time elevation (PIM, temporary roles)
- Access reviews on a schedule

Sheet 06

Workload Identity & Federation

IAM roles · managed identities · service accounts · short-lived credentials · OIDC federation from CI · why long-lived keys are risky

CREDENTIALS FOR CODE, WITHOUT SECRETS

Applications need identities too. Instead of pasting an access key into config, attach an identity to the compute resource: an **IAM role** on AWS, a **managed identity** on Azure, a **service account** on Google Cloud. The platform hands out short-lived tokens automatically. Outside the cloud (CI pipelines, other clouds) use **OIDC federation**: the provider trusts tokens from GitHub or another identity provider.

KEY POINTS

- Tokens expire in about an hour and refresh themselves
- SDKs find these credentials automatically (default credential chains)
- Federation means no stored secret in GitHub at all
- A leaked long-lived key works until someone notices and revokes it

SAME CODE, NO KEYS (SDK DEFAULT CHAINS)

```
# AWS: boto3 picks up the role from the environment / instance profile
import boto3
s3 = boto3.client("s3")

# Azure: DefaultAzureCredential uses the managed identity in Azure
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient
blobs = BlobServiceClient("https://demostore.blob.core.windows.net",
                          credential=DefaultAzureCredential())

# Google Cloud: Application Default Credentials use the service account
from google.cloud import storage
gcs = storage.Client()
```

CODE EXPLAINED

- → No credentials appear in the code: the default chain finds them
- → On a laptop the same code uses your CLI login instead
- → `id-token: write` lets the GitHub job request an OIDC token
- → Each cloud trusts that token through a role, app registration or pool

GITHUB ACTIONS OIDC LOGINS

```
permissions: { id-token: write, contents: read }
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with: { role-to-assume: "arn:aws:iam::111122223333:role/gh-deploy",
            aws-region: us-east-1 }
  - uses: azure/login@v2
    with: { client-id: ..., tenant-id: ..., subscription-id: ... }
  - uses: google-github-actions/auth@v2
    with: { workload_identity_provider: "projects/123/locations/global/...",
            service_account: "deploy@app-prod.iam.gserviceaccount.com" }
```

ACROSS THE CLOUDS

- ↔ **AWS** IAM roles, instance profiles, OIDC providers (aws-07)
- ↔ **Azure** managed identities, federated credentials (az-07)
- ↔ **Google Cloud** service accounts, Workload Identity Federation (gcp-07)

RULE OF THUMB

- ☞ If you are about to create an access key, look for the identity-based option first
- ☞ One identity per workload, with only the roles that workload needs
- ☞ Federate CI; never store cloud keys as repository secrets

WORKLOAD IDENTITY

NEED	AWS	AZURE	GOOGLE CLOUD
VM / container	Instance profile, task role	Managed identity	Attached service account
Kubernetes pod	EKS Pod Identity	AKS workload identity	GKE Workload Identity Federation
CI pipeline	OIDC provider + role	Federated credential	Workload Identity Federation
Local dev	SSO profile	az login	gcloud auth application-default login

DO & DON'T

- ✓ **Do** Use the SDK default credential chain in all code
- ✓ **Do** Scope federation trust to one repo and branch / environment
- ✗ **Don't** Commit a key file or `.env` with cloud secrets
- ✗ **Don't** Reuse one powerful identity for every service

LOCAL TESTING

- ⌂ Laptop: `aws sso login`, `az login`, `gcloud auth application-default login` feed the same default chains
- ⌂ LocalStack and Azurite accept dummy credentials, so no real keys are needed
- ⌂ Use `act` to dry-run GitHub workflows (OIDC steps need the real runner)

PRODUCTION CHECKLIST

- ☐ Zero long-lived access keys for workloads (alert on any that appear)
- ☐ Federation trust conditions pinned to repo, branch and environment
- ☐ Key creation blocked by policy where the cloud supports it

Secrets & Encryption

PROTECTING PASSWORDS, KEYS AND DATA

Store passwords, API tokens and connection strings in a **secret store** (AWS Secrets Manager, Azure Key Vault, Google Secret Manager) and read them at runtime with the workload's identity. Encryption keys live in a **key management service** (AWS KMS, Key Vault keys, Cloud KMS), which never reveals the master key. Data is encrypted at rest by default on all three; you choose who manages the key.

KEY POINTS

- Secrets are versioned: rotate by adding a new version
- Envelope encryption: a data key encrypts the data, the master key encrypts the data key
- Provider-managed keys are the default; customer-managed keys (CMK / CMEK) add control and audit
- TLS protects data in transit; enforce HTTPS-only on storage and APIs

STORE AND READ A SECRET

```
# AWS Secrets Manager
aws secretsmanager create-secret --name prod/db-password \
    --secret-string 'S3cure!pass'
aws secretsmanager get-secret-value --secret-id prod/db-password \
    --query SecretString --output text

# Azure Key Vault
az keyvault secret set --vault-name kv-shop-prod \
    --name db-password --value 'S3cure!pass'
az keyvault secret show --vault-name kv-shop-prod \
    --name db-password --query value -o tsv

# Google Secret Manager
printf 'S3cure!pass' | gcloud secrets create db-password --data-file=-
gcloud secrets versions access latest --secret=db-password

# Encrypt a small value with each KMS
aws kms encrypt --key-id alias/app --plaintext fileb://token.txt
az keyvault key encrypt --vault-name kv-shop-prod -n app-key \
    --algorithm RSA-OAEP-256 --value <base64>
gcloud kms encrypt --key=app --keyring=main --location=global \
    --plaintext-file=token.txt --ciphertext-file=token.enc
```

CODE EXPLAINED

- → All three keep secret history as versions
- → Reading a secret is an authorized, logged API call
- → Shell history can capture secrets; real scripts read from files or stdin
- → KMS encrypt calls are for small payloads (keys, tokens), not files

ACROSS THE CLOUDS

- ↔ **AWS** Secrets Manager, Parameter Store, KMS (aws-08)
- ↔ **Azure** Key Vault secrets, keys and certificates (az-08)
- ↔ **Google Cloud** Secret Manager, Cloud KMS (gcp-08)

RULE OF THUMB

- ☞ Secrets in a secret store, read at startup or through platform references
- ☞ Default encryption is fine for most data; use customer-managed keys when compliance asks
- ☞ Automate rotation for database passwords

SERVICES

NEED	AWS	AZURE	GOOGLE CLOUD
Secrets	Secrets Manager / Parameter Store	Key Vault secrets	Secret Manager
Encryption keys	KMS	Key Vault keys / Managed HSM	Cloud KMS / Cloud HSM
TLS certificates	ACM	Key Vault certificates / App Service certs	Certificate Manager
Platform reference	ECS/Lambda secrets	Key Vault references	Cloud Run --set-secrets

DO & DON'T

- ✓ **Do** Inject secrets at runtime through platform integrations
- ✓ **Do** Turn on soft delete / deletion protection for vaults and keys
- ✗ **Don't** Put secrets in environment variables baked into images or IaC files
- ✗ **Don't** Log request bodies that may contain tokens

LOCAL TESTING

- ⌂ Use a local `.env` that is gitignored, loaded only on your machine
- ⌂ LocalStack emulates Secrets Manager and KMS; Azurite does not cover Key Vault (use a dev vault)
- ⌂ Scan commits with gitleaks or GitHub secret scanning before pushing

PRODUCTION CHECKLIST

- ☐ Secrets read by managed identity, never stored in code or pipelines
- ☐ Rotation scheduled and tested
- ☐ Deletion protection on vaults, keys and secrets

Security Baseline & Guardrails

audit logging · org-level policies · public-access blocks · threat detection services · tagging & ownership · the first-day checklist

THE SETTINGS TO TURN ON FIRST

A few controls prevent most cloud incidents: **audit logs** of every API call, **organization policies** that forbid risky configurations, **public-access blocks** on storage, **threat detection** that flags odd behavior, and **tags** so every resource has an owner. All three providers offer each piece; turning them on is cheap compared with cleaning up a breach.

KEY POINTS

- Audit logs: CloudTrail, Azure Activity Log, Cloud Audit Logs
- Guardrails: Service Control Policies, Azure Policy, Organization Policy constraints
- Posture and threats: Security Hub + GuardDuty, Defender for Cloud, Security Command Center
- Most breaches start with an exposed key or a public bucket

TURNING ON GUARDRAILS

```
# AWS: block public S3 access for the whole account
aws s3control put-public-access-block --account-id 111122223333 \
    --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,\
RestrictPublicBuckets=true
aws guardduty create-detector --enable

# Azure: assign a built-in policy that denies public blob access
DEF=$(az policy definition list --query \
  "[?displayName=='Storage account public access should be disallowed'].name" \
  -o tsv)
az policy assignment create --name deny-public-blob --policy $DEF \
    --scope /subscriptions/<sub-id>

# Google Cloud: enforce public access prevention org-wide
gcloud resource-manager org-policies enable-enforce \
    storage.publicAccessPrevention --organization=123456789012
gcloud resource-manager org-policies enable-enforce \
    iam.disableServiceAccountKeyCreation --organization=123456789012
```

CODE EXPLAINED

- → The AWS account-level block overrides any bucket that tries to go public
- → Azure looks up the built-in policy by display name, then assigns it
- → Google org policies apply to every project under the organization
- → Blocking service account key creation forces identity-based access

ACROSS THE CLOUDS

- ↔ **AWS** CloudTrail, GuardDuty, Security Hub, Config, SCPs (aws-09)
- ↔ **Azure** Activity Log, Defender for Cloud, Azure Policy (az-09)
- ↔ **Google Cloud** Cloud Audit Logs, Security Command Center, Organization Policy (gcp-06, gcp-09)

RULE OF THUMB

- ☞ Turn guardrails on before teams start building, not after
- ☞ Prefer deny policies at the organization over review checklists
- ☞ Every resource needs an owner tag; untagged resources get questions

FIRST-DAY CHECKLIST

CONTROL	AWS	AZURE	GOOGLE CLOUD
Audit log kept 1 yr+	CloudTrail org trail	Activity Log → Log Analytics	Audit Logs → log bucket
MFA for admins	IAM Identity Center	Conditional Access	2-Step Verification
Block public storage	S3 Block Public Access	Policy: no anonymous blob	Public access prevention
Threat detection	GuardDuty	Defender for Cloud	Security Command Center
Budget alert	AWS Budgets	Cost Management budget	Billing budget

DO & DON'T

- ✓ **Do** Send audit logs to a separate, locked-down account or project
- ✓ **Do** Review the security posture score weekly
- ✗ **Don't** Disable a guardrail to make a deploy pass and forget to turn it back on
- ✗ **Don't** Ignore findings until an audit forces you to look

LOCAL TESTING

- ⌂ Scan IaC before deploy with Checkov, tfsec / Trivy or KICS
- ⌂ Prowler (AWS, Azure, Google Cloud) audits a live account against benchmarks
- ⌂ Use a sandbox to test that a policy blocks what you expect

PRODUCTION CHECKLIST

- ☐ CIS benchmark checks enabled in the posture tool
- ☐ Audit logs retained per compliance rules and immutable
- ☐ Alerts for root / Global Administrator sign-ins and policy changes

Virtual Networks & Subnets

CIDR planning · public vs private subnets · route tables · internet & NAT gateways · regional (AWS, Azure) vs global (Google Cloud) networks · peering

YOUR PRIVATE NETWORK IN THE CLOUD

A virtual network (**VPC** on AWS and Google Cloud, **VNet** on Azure) is an isolated private IP space you carve into **subnets**. A subnet is "public" when its route table sends internet traffic to an internet gateway, "private" when outbound traffic goes through a **NAT gateway** or nowhere. AWS and Azure networks are regional; a Google Cloud VPC is **global** with regional subnets.

KEY POINTS

- Use private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- Plan CIDRs so networks you may connect later do not overlap
- Providers reserve some addresses per subnet (AWS 5, Azure 5, Google Cloud 4)
- Keep databases and app servers in private subnets

ONE NETWORK, TWO SUBNETS

```
# AWS: VPC + public and private subnets in one AZ
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-subnet --vpc-id vpc-0abc --cidr-block 10.0.1.0/24 \
    --availability-zone us-east-1a
aws ec2 create-subnet --vpc-id vpc-0abc --cidr-block 10.0.2.0/24 \
    --availability-zone us-east-1a

# Azure: VNet with two subnets
az network vnet create -g rg-net -n vnet-app \
    --address-prefixes 10.0.0.0/16 \
    --subnet-name snet-web --subnet-prefixes 10.0.1.0/24
az network vnet subnet create -g rg-net --vnet-name vnet-app \
    -n snet-db --address-prefixes 10.0.2.0/24

# Google Cloud: custom-mode VPC, subnets per region
gcloud compute networks create vpc-app --subnet-mode=custom
gcloud compute networks subnets create web-us \
    --network=vpc-app --region=us-central1 --range=10.0.1.0/24
gcloud compute networks subnets create web-eu \
    --network=vpc-app --region=europe-west1 --range=10.1.1.0/24
```

CODE EXPLAINED

- AWS subnets live in one AZ; Azure and Google Cloud subnets span the region's zones
- One Google Cloud VPC holds subnets in many regions
- A /24 gives 256 addresses minus the reserved ones
- Routes, gateways and firewalls come next (sheets 10–11)

ACROSS THE CLOUDS

- **AWS** VPC, subnets, route tables, Internet Gateway (aws-10)
- **Azure** VNet, subnets, route tables (az-10)
- **Google Cloud** global VPC, regional subnets (gcp-10)

RULE OF THUMB

- Give each environment a /16 from a company-wide plan
- Size subnets generously: resizing later is painful
- Default to private subnets; expose only load balancers

PIECES

PIECE	AWS	AZURE	GOOGLE CLOUD
Network	VPC (regional)	VNet (regional)	VPC (global)
Subnet scope	One AZ	Region	Region
Internet path	Internet Gateway + route	Public IP / Load Balancer	External IP + default route
Private outbound	NAT Gateway	NAT Gateway	Cloud NAT
Connect networks	Peering / Transit Gateway	Peering / Virtual WAN	Peering / NCC

CIDR SIZES

PREFIX	ADDRESSES
/16	65,536
/20	4,096
/24	256
/27	32
/28	16

DO & DON'T

- ✓ **Do** Write the IP plan down before creating the first network
- ✓ **Do** Put each tier (web, app, data) in its own subnet
- ✗ **Don't** Use the default network for production
- ✗ **Don't** Pick 10.0.0.0/16 everywhere and discover overlaps when peering

LOCAL TESTING

- Docker networks and kind / minikube give you subnets and routing to experiment with
- Plan ranges with `ipcalc` or an online subnet calculator
- Bitwise set, sheet on IP subnets and CIDR, covers the math

PRODUCTION CHECKLIST

- Non-overlapping CIDRs across all environments and on-premises
- Flow logs enabled for troubleshooting and audit
- Network created by IaC with the plan documented

Firewalls & Private Access

security groups · NSGs · VPC firewall rules · stateful vs stateless · private endpoints / PrivateLink / Private Service Connect · bastions & identity-aware access

CONTROLLING TRAFFIC IN AND OUT

Each cloud filters traffic with stateful rules: **security groups** on AWS, **network security groups (NSGs)** on Azure, **VPC firewall rules / policies** on Google Cloud. To reach managed services without the public internet, use **private endpoints** (PrivateLink, Azure Private Endpoint, Private Service Connect). For admin access, use identity-aware tunnels instead of open SSH ports.

KEY POINTS

- Stateful: replies to allowed traffic are allowed automatically
- Reference groups / tags / service accounts instead of IP addresses
- Private endpoints give a service a private IP inside your network
- Session Manager, Azure Bastion and IAP replace public SSH / RDP

ALLOW HTTPS IN, NOTHING ELSE

```
# AWS security group: 443 from anywhere
aws ec2 create-security-group --group-name web-sg \
    --description "web" --vpc-id vpc-0abc
aws ec2 authorize-security-group-ingress --group-id sg-0123 \
    --protocol tcp --port 443 --cidr 0.0.0.0/0

# Azure NSG rule on the web subnet
az network nsg create -g rg-net -n nsg-web
az network nsg rule create -g rg-net --nsg-name nsg-web \
    -n allow-https --priority 100 --direction Inbound \
    --access Allow --protocol Tcp --destination-port-ranges 443

# Google Cloud firewall rule targeting tagged VMs
gcloud compute firewall-rules create allow-https --network=vpc-app \
    --direction=INGRESS --allow=tcp:443 \
    --source-ranges=0.0.0.0/0 --target-tags=web

# Admin access without opening port 22
aws ssm start-session --target i-0abc123
az network bastion ssh -n bas-app -g rg-net --target-resource-id <vm-id> \
    --auth-type AAD
gcloud compute ssh web-1 --zone=us-central1-a --tunnel-through-iap
```

CODE EXPLAINED

- All three deny inbound by default, so you add only what you need
- Azure rules run in priority order (lower number first)
- Google Cloud rules apply to VMs by network tag or service account
- The last three lines open admin sessions through identity, not open ports

ACROSS THE CLOUDS

- **AWS** security groups, NACLs, PrivateLink (aws-11, aws-12)
- **Azure** NSGs, Azure Firewall, Private Endpoints (az-11, az-12)
- **Google Cloud** firewall rules & policies, Private Service Connect, IAP (gcp-11, gcp-12)

RULE OF THUMB

- Only load balancers face the internet; everything else is private
- Use private endpoints for databases, storage and secret stores in production
- Never open 22 / 3389 to 0.0.0.0/0

COMPARISON

FEATURE	AWS	AZURE	GOOGLE CLOUD
Instance firewall	Security group	NSG (subnet or NIC)	Firewall rule (tag / SA)
Subnet filter	NACL (stateless)	NSG on subnet	Firewall policy
Central firewall	Network Firewall	Azure Firewall	Cloud NGFW
Private service access	PrivateLink / endpoints	Private Endpoint	Private Service Connect
Admin access	Session Manager	Bastion	IAP TCP forwarding

DO & DON'T

- ✓ **Do** Reference security groups / ASGs / tags rather than IP lists
- ✓ **Do** Log denied traffic while you tune rules
- ✗ **Don't** Allow all outbound and inbound "for now"
- ✗ **Don't** Expose a database on a public IP, even with a password

LOCAL TESTING

- Test rules in a sandbox with `nc -zv host 443` or `curl -v` from inside and outside
- Reachability Analyzer (AWS), Network Watcher (Azure) and Connectivity Tests (Google Cloud) explain blocked paths
- Docker Compose networks mimic tiered isolation locally

PRODUCTION CHECKLIST

- No public IPs on app servers or databases
- Private endpoints plus private DNS for PaaS services
- Flow logs and firewall logs sent to the central log store

Load Balancing & DNS

L4 vs L7 · health checks · global vs regional load balancers · managed DNS zones · routing policies · TLS certificates

GETTING USERS TO HEALTHY SERVERS

A **load balancer** spreads requests across healthy back ends and gives users one address. **Layer 4** balancers forward TCP/UDP; **layer 7** balancers understand HTTP and route by host or path, terminate TLS and add a WAF. **Managed DNS** maps names to those addresses and can route by latency, weight or health.

KEY POINTS

- Health checks remove failed instances automatically
- Google Cloud's external Application Load Balancer is global: one anycast IP worldwide
- AWS and Azure L7 balancers are regional; global front doors are separate services
- Managed certificates renew automatically

AN HTTP LOAD BALANCER, SKETCHED

```
# AWS Application Load Balancer + target group
aws elbv2 create-load-balancer --name web-alb \
    --subnets subnet-a subnet-b --security-groups sg-0123
aws elbv2 create-target-group --name web-tg --protocol HTTP \
    --port 8080 --vpc-id vpc-0abc --health-check-path /health

# Azure Application Gateway (L7) in its own subnet
az network application-gateway create -g rg-net -n agw-web \
    --sku Standard_v2 --capacity 2 --vnet-name vnet-app \
    --subnet snet-agw --public-ip-address pip-agw --priority 100

# Google Cloud: health check + backend service (global ALB)
gcloud compute health-checks create http web-hc --port=8080 \
    --request-path=/health
gcloud compute backend-services create web-be --global \
    --protocol=HTTP --health-checks=web-hc

# DNS record pointing a name at the balancer
aws route53 change-resource-record-sets --hosted-zone-id Z123 \
    --change-batch file://alias-record.json
az network dns record-set a add-record -g rg-dns -z example.com \
    -n www -a 20.42.0.10
gcloud dns record-sets create www.example.com. --zone=example \
    --type=A --ttl=300 --rrdatas=34.120.0.10
```

CODE EXPLAINED

- → Every balancer needs a health check path that returns 200 when healthy
- → Azure Application Gateway needs a dedicated subnet
- → Google Cloud builds an ALB from parts: health check, backend service, URL map, proxy, forwarding rule
- → AWS alias records point at AWS resources without a fixed IP

ACROSS THE CLOUDS

- ↔ **AWS** ALB / NLB, Route 53 (aws-13, aws-17)
- ↔ **Azure** Load Balancer, Application Gateway, Azure DNS, Traffic Manager (az-13, az-17)
- ↔ **Google Cloud** Cloud Load Balancing, Cloud DNS (gcp-13, gcp-17)

RULE OF THUMB

- ☞ Use L7 for web apps and APIs, L4 for raw TCP / UDP and extreme throughput
- ☞ Keep DNS TTLs low (60–300 s) on records you may fail over
- ☞ Terminate TLS at the balancer with managed certificates

LOAD BALANCER TYPES

TYPE	AWS	AZURE	GOOGLE CLOUD
L7 regional	ALB	Application Gateway	Regional Application LB
L7 global	CloudFront + ALB / Global Accelerator	Front Door	Global external Application LB
L4	NLB	Azure Load Balancer	Network LB (proxy / passthrough)
DNS routing	Route 53 policies	Traffic Manager	Cloud DNS routing policies

DO & DON'T

- ✓ **Do** Make health checks test real dependencies lightly (not just "process alive")
- ✓ **Do** Redirect HTTP to HTTPS at the balancer
- ✗ **Don't** Point DNS straight at single VM IPs in production
- ✗ **Don't** Set a 24-hour TTL on a record you might need to change quickly

LOCAL TESTING

- ⌂ Run nginx, HAProxy or Traefik in Docker Compose to practice routing and health checks
- ⌂ `dig +short www.example.com` and `curl -v --resolve` test DNS and TLS
- ⌂ kind / minikube Ingress controllers mimic L7 routing

PRODUCTION CHECKLIST

- ☐ Balancer across at least two zones with health checks tuned
- ☐ Managed TLS certificates with monitored expiry
- ☐ WAF on public HTTP endpoints

CDN & Edge

caching & TTLs · origins · invalidation · WAF at the edge · CloudFront / Front Door / Cloud CDN · static site hosting pattern

SERVING CONTENT CLOSE TO USERS

A **content delivery network** caches responses at hundreds of edge locations so users get static files (and cacheable API responses) from nearby. You configure **origins** (a bucket, load balancer or app), cache rules and TTLs, and optionally a WAF. The classic pattern is a static site in object storage behind the CDN with a managed certificate.

KEY POINTS

- Cache keys decide what counts as the same object (path, query, headers)
- Use versioned file names (`app.3f9a.js`) instead of frequent invalidations
- Keep the origin private: only the CDN can read it
- Edge services also absorb DDoS traffic

STATIC SITE BEHIND A CDN

```
# AWS: upload, then invalidate after a deploy
aws s3 sync ./dist s3://site-bucket --delete
aws cloudfront create-invalidation --distribution-id E1ABC \
    --paths "/index.html"

# Azure: Front Door profile + endpoint, then purge after deploy
az afd profile create -g rg-web --profile-name afd-site \
    --sku Standard_AzureFrontDoor
az afd endpoint create -g rg-web --profile-name afd-site \
    --endpoint-name site
az afd endpoint purge -g rg-web --profile-name afd-site \
    --endpoint-name site --content-paths "/index.html"

# Google Cloud: backend bucket with CDN, invalidate a path
gcloud compute backend-buckets create site-be \
    --gcs-bucket-name=site-bucket --enable-cdn
gcloud compute url-maps invalidate-cdn-cache site-map \
    --path="/index.html"
```

CODE EXPLAINED

- → `sync --delete` mirrors the build folder into the bucket
- → Invalidate only the entry HTML when asset names are versioned
- → Azure Front Door is the CDN and global L7 balancer in one
- → Google Cloud CDN attaches to a load balancer backend bucket or service

ACROSS THE CLOUDS

- ↔ **AWS** CloudFront, ACM, Origin Access Control (aws-14)
- ↔ **Azure** Front Door Standard / Premium (az-14)
- ↔ **Google Cloud** Cloud CDN, Certificate Manager (gcp-14)

RULE OF THUMB

- ☞ Long TTLs on hashed assets, short TTL on `index.html`
- ☞ Lock the origin so it only accepts CDN traffic
- ☞ Put a WAF on the edge for public apps

CDN SERVICES

FEATURE	AWS	AZURE	GOOGLE CLOUD
CDN	CloudFront	Front Door	Cloud CDN
Private origin	Origin Access Control	Private Link origin	Private bucket + signed requests
Edge compute	CloudFront Functions / Lambda@Edge	Rules engine	Service extensions
WAF	AWS WAF	Front Door WAF	Cloud Armor

CACHE-CONTROL HEADERS

HEADER	USE
`max-age=31536000, immutable`	Hashed assets
`no-cache`	Revalidate every time (HTML)
`private`	Never cache at CDN
`s-maxage=60`	CDN-only TTL

DO & DON'T

- ✓ **Do** Fingerprint asset file names in the build
- ✓ **Do** Set Cache-Control on upload, not only in CDN rules
- ✗ **Don't** Invalidate `/*` on every deploy (slow and sometimes billed)
- ✗ **Don't** Leave the storage bucket publicly readable behind the CDN

LOCAL TESTING

- ⌂ `npx serve` or `python -m http.server` previews the static site
- ⌂ Check headers with `curl -I https://site/app.js`
- ⌂ Use the HTTP set's caching sheet for header details

PRODUCTION CHECKLIST

- ☐ Managed certificate and HTTPS redirect at the edge
- ☐ Origin locked to the CDN only
- ☐ Cache hit ratio monitored; WAF rules in prevention mode

Virtual Machines & Autoscaling

instance families & sizes · images · startup scripts · disks · scale groups (ASG / VMSS / MIG) · Arm instances

RENTING SERVERS BY THE SECOND

A virtual machine is a slice of a physical server with your choice of CPU, memory, disk and image. Families target workloads (general purpose, compute, memory, GPU), and **Arm** families (Graviton, Cobalt, Axion) are usually cheaper per unit of work. Put identical VMs in a **scale group** (Auto Scaling group, VM Scale Set, managed instance group) to add and remove machines with load.

KEY POINTS

- Billing is per second (AWS Linux, Google Cloud) or per minute (Azure) while running
- Stopped VMs still pay for disks and reserved IPs
- Startup scripts / cloud-init configure a VM on first boot
- Treat VMs as cattle: rebuild from an image instead of fixing in place

LAUNCH A SMALL LINUX VM

```
# AWS
P=/aws/service/ami-amazon-linux-latest
aws ec2 run-instances \
    --image-id resolve:ssm:$P/al2023-ami-kernel-default-arm64 \
    --instance-type t4g.small --subnet-id subnet-a \
    --user-data file://init.sh --metadata-options HttpTokens=required

# Azure
az vm create -g rg-app -n vm-web-1 --image Ubuntu2404 \
    --size Standard_B2s --custom-data init.sh \
    --generate-ssh-keys --public-ip-address ""

# Google Cloud
gcloud compute instances create web-1 --zone=us-central1-a \
    --machine-type=e2-small --image-family=debian-12 \
    --image-project=debian-cloud --no-address \
    --metadata-from-file=startup-script=init.sh
```

CODE EXPLAINED

- → Each command picks a small size, an image and a first-boot script
- → AWS resolves the latest Amazon Linux image from a public SSM parameter
- → `HttpTokens=required` enforces IMDSv2 on AWS
- → Empty public IP / `--no-address` keeps the VM private

ACROSS THE CLOUDS

- ↔ **AWS** EC2, AMIs, Auto Scaling groups (aws-15, aws-16, aws-18)
- ↔ **Azure** VMs, managed disks, VM Scale Sets (az-15, az-16, az-18)
- ↔ **Google Cloud** Compute Engine, managed instance groups (gcp-15, gcp-16, gcp-18)

RULE OF THUMB

- ☞ Choose the smallest size that passes load tests, then scale out
- ☞ Try Arm first for Linux workloads: often 20–40% better price-performance
- ☞ Bake images with Packer for faster, repeatable boots

FAMILIES (EXAMPLES)

WORKLOAD	AWS	AZURE	GOOGLE CLOUD
Burstable / small	t4g, t3	B-series	e2
General purpose	m7g, m7i	Dsv5 / Dpsv6	n4, c4
Compute heavy	c7g, c7i	Fsv2	c3, c4
Memory heavy	r7g	Esv5	m3, n4-highmem
Arm	Graviton	Cobalt	Axion (c4a)

DO & DON'T

- ✓ **Do** Keep VMs stateless; store data in managed disks, databases or object storage
- ✓ **Do** Use scale groups even for one instance (self-healing)
- ✗ **Don't** SSH in to patch production servers by hand
- ✗ **Don't** Leave dev VMs running 24/7

LOCAL TESTING

- ⌂ Multipass, Vagrant or a local VM runs the same cloud-init script
- ⌂ Test startup scripts in a container first (`docker run -it ubuntu`)
- ⌂ Packer builds images locally before pushing to a cloud

PRODUCTION CHECKLIST

- □ Images patched and rebuilt on a schedule
- □ Instances spread across zones in a scale group with health checks
- □ No public IPs; admin access through Session Manager, Bastion or IAP

Sheet 14

Containers & Managed Kubernetes

registries · serverless containers (Fargate, Container Apps, Cloud Run) · EKS / AKS / GKE · control plane vs nodes · pointer to Docker & Kubernetes sets

RUNNING CONTAINERS WITHOUT RUNNING SERVERS

Package the app as a container image, push it to a **registry**, and pick how much of the platform you want to manage. **Serverless containers** (AWS Fargate with ECS or App Runner, Azure Container Apps, Google Cloud Run) run an image with autoscaling and no cluster to maintain. **Managed Kubernetes** (EKS, AKS, GKE) gives you the full Kubernetes API with the control plane run for you.

KEY POINTS

- Registries: ECR, Azure Container Registry, Artifact Registry
- Cloud Run and Container Apps can scale to zero; idle cost is near nothing
- Cluster fees: EKS and GKE bill per cluster-hour (a GKE credit covers one cluster); AKS Free tier has none
- See the Docker and Kubernetes sets for the tools themselves

BUILD, PUSH, RUN

```
# Push to each registry
aws ecr get-login-password | docker login --username AWS \
    --password-stdin 111122223333.dkr.ecr.us-east-1.amazonaws.com
az acr login --name shopacr
gcloud auth configure-docker us-central1-docker.pkg.dev
docker push us-central1-docker.pkg.dev/app-prod/web/api:1.4.0

# Serverless containers
az containerapp up -n api -g rg-app \
    --image shopacr.azurecr.io/api:1.4.0 --ingress external \
    --target-port 8080
gcloud run deploy api --region=us-central1 \
    --image=us-central1-docker.pkg.dev/app-prod/web/api:1.4.0

# Managed Kubernetes: create a cluster, get kubectl credentials
eksctl create cluster --name shop --region us-east-1
az aks create -g rg-app -n aks-shop --generate-ssh-keys
gcloud container clusters create-auto shop --region=us-central1
aws eks update-kubeconfig --name shop
az aks get-credentials -g rg-app -n aks-shop
gcloud container clusters get-credentials shop --region=us-central1
```

CODE EXPLAINED

→ Each registry has its own login helper for Docker

→ Cloud Run and Container Apps deploy straight from an image

→ Autopilot (GKE) and AKS Automatic manage nodes for you

→ After `get-credentials`, plain `kubectl` works the same on all three

ACROSS THE CLOUDS

↔ **AWS** ECR, ECS / Fargate, EKS (aws-25, aws-26, aws-27)

↔ **Azure** ACR, Container Apps, AKS (az-25, az-26, az-27)

↔ **Google Cloud** Artifact Registry, Cloud Run, GKE (gcp-25, gcp-20, gcp-26)

RULE OF THUMB

☞ Start with serverless containers; move to Kubernetes when you need its ecosystem

☞ Pin images by tag and digest; never deploy `latest`

☞ One cluster can host many apps; one Cloud Run service hosts one

OPTIONS

NEED	AWS	AZURE	GOOGLE CLOUD
Registry	ECR	ACR	Artifact Registry
Simple container service	App Runner / ECS on Fargate	Container Apps	Cloud Run
Managed Kubernetes	EKS	AKS	GKE Standard
Nodes managed for you	EKS Auto Mode	AKS Automatic	GKE Autopilot
Single container quick	ECS task	Container Instances	Cloud Run job

DO & DON'T

✓ **Do** Scan images in the registry and fail builds on critical findings

✓ **Do** Run containers as non-root with read-only file systems where possible

✗ **Don't** Store secrets in the image

✗ **Don't** Build a Kubernetes platform for a single small service

LOCAL TESTING

⌂ Docker Desktop, Podman or Rancher Desktop run the same image

⌂ kind / k3d / minikube give a local Kubernetes cluster

⌂ Cloud Run: `gcloud beta code dev` or plain `docker run -p 8080:8080`

PRODUCTION CHECKLIST

☐ Images signed or from a trusted registry only

☐ Resource requests / limits and health probes defined

☐ Autoscaling bounds set (min for latency, max for cost)

Serverless Functions

CODE THAT RUNS WHEN SOMETHING HAPPENS

A serverless function is a handler the platform runs in response to an event: an HTTP request, a new file in a bucket, a queue message or a timer. You pay per invocation and execution time, and it scales to zero when idle. The trade-offs are **cold starts** on the first request after idle time, execution time limits, and less control over the runtime.

KEY POINTS

- AWS Lambda, Azure Functions and Cloud Run functions (formerly Cloud Functions)
- Triggers are wired by the platform; your code just receives the event
- Keep handlers small; reuse clients across invocations
- Long work goes to queues, workflows or containers

AN HTTP FUNCTION ON EACH CLOUD

```python
# AWS Lambda (Python)
def handler(event, context):
    name = (event.get("queryStringParameters") or {}).get("name", "world")
    return {"statusCode": 200, "body": f"hello {name}"}

# Azure Functions (Python v2 programming model)
import azure.functions as func
app = func.FunctionApp()
@app.route(route="hello", auth_level=func.AuthLevel.ANONYMOUS)
def hello(req: func.HttpRequest) -> func.HttpResponse:
    return func.HttpResponse(f"hello {req.params.get('name', 'world')}")

# Cloud Run functions (Functions Framework)
import functions_framework
@functions_framework.http
def hello(request):
    return f"hello {request.args.get('name', 'world')}"
```

CODE EXPLAINED

- → Each platform wraps the request in its own event / request object
- → Azure declares routes with decorators in `function_app.py`
- → The Functions Framework makes the Google function a normal Flask-style handler
- → Deploy commands package and upload the code

DEPLOY

```
aws lambda create-function --function-name hello --runtime python3.13 \
    --handler app.handler --zip-file fileb://app.zip --role <role-arn>
func azure functionapp publish func-hello
gcloud functions deploy hello --gen2 --runtime=python313 \
    --trigger-http --entry-point=hello --region=us-central1
```

ACROSS THE CLOUDS

- ↔ **AWS** Lambda (aws-20, aws-21)
- ↔ **Azure** Azure Functions (az-20, az-21)
- ↔ **Google Cloud** Cloud Run functions (gcp-21)

RULE OF THUMB

- ☞ Use functions for glue, webhooks, scheduled jobs and event processing
- ☞ Move to containers when a function needs more than a few minutes or custom runtimes
- ☞ Make handlers idempotent: events can arrive twice

COMMON LIMITS (VERIFY PER PLAN)

LIMIT	LAMBDA	AZURE FUNCTIONS	CLOUD RUN FUNCTIONS
Max run time	15 min	Flex: 30 min default, can raise · Consumption: 10 min	60 min HTTP · 9 min events
Memory	128 MB–10 GB	Flex: 512 MB–4 GB instances	Up to 32 GiB
Scale to zero	Yes	Yes (Flex / Consumption)	Yes
Concurrency per instance	1	Many	Up to 1,000

DO & DON'T

- ✓ **Do** Create SDK clients outside the handler so warm calls reuse them
- ✓ **Do** Send failures to a dead-letter queue
- ✗ **Don't** Chain functions synchronously into a call stack
- ✗ **Don't** Hold database connections without a pool or proxy

LOCAL TESTING

- ⌂ AWS SAM: `sam local invoke` / `sam local start-api`
- ⌂ Azure Functions Core Tools: `func start`
- ⌂ Functions Framework: `functions-framework --target=hello`

PRODUCTION CHECKLIST

- ☐ Timeouts and memory tuned from real measurements
- ☐ Dead-letter queues and alerts on errors and throttles
- ☐ Concurrency limits set to protect downstream databases

Choosing Compute

VMs vs containers vs functions vs PaaS · control vs convenience · cost shape · decision table across the three providers

MATCHING THE WORKLOAD TO THE SERVICE

Every cloud offers a ladder from full control to full convenience: **VMs**, **managed Kubernetes**, **serverless containers**, **app platforms** and **functions**. Climb as high as the workload allows: each step up removes servers to patch and scaling to tune, at the cost of limits and some lock-in. Cost shape differs too: VMs bill while running, serverless bills per request.

KEY POINTS

- Steady 24/7 load is often cheapest on VMs or containers with commitments
- Spiky or idle-heavy load is cheapest on scale-to-zero services
- Kubernetes pays off with many services and a team to run it
- Batch jobs fit spot VMs or job services

ONE CONTAINER, THREE PLATFORMS

```
IMAGE=registry.example.com/shop/api:1.4.0

# Serverless container: the default choice for HTTP services
gcloud run deploy api --image=$IMAGE --region=us-central1 \
    --min-instances=0 --max-instances=20
az containerapp create -n api -g rg-app --environment cae-app \
    --image $IMAGE --min-replicas 0 --max-replicas 20 \
    --ingress external --target-port 8080
aws apprunner create-service --service-name api \
    --source-configuration file://apprunner.json

# Kubernetes: when you already run a cluster
kubectl create deployment api --image=$IMAGE --replicas=3
kubectl expose deployment api --port=80 --target-port=8080

# VM: when you need the OS (agents, GPUs, licensing)
gcloud compute instances create api-vm --zone=us-central1-a \
    --metadata-from-file=startup-script=run-container.sh
```

CODE EXPLAINED

- → The same image runs on every rung of the ladder
- → Min 0 lets serverless containers scale to zero when idle
- → App Runner takes its settings from a JSON source configuration
- → VMs make sense when the workload needs the operating system

ACROSS THE CLOUDS

- ↔ **AWS** Choosing Compute (aws-19)
- ↔ **Azure** Choosing Compute (az-19)
- ↔ **Google Cloud** Choosing Compute (gcp-19)

RULE OF THUMB

- ☞ Default: serverless containers for web services, functions for events
- ☞ Pick Kubernetes for a platform, not for one app
- ☞ Pick VMs for legacy apps, GPUs, special licensing or OS-level agents

DECISION TABLE

NEED	AWS	AZURE	GOOGLE CLOUD
HTTP service, low ops	App Runner / ECS Fargate	Container Apps / App Service	Cloud Run
Event glue	Lambda	Functions	Cloud Run functions
Many microservices	EKS	AKS	GKE
Full OS control	EC2	Virtual Machines	Compute Engine
Batch	AWS Batch	Azure Batch / Container Apps jobs	Batch / Cloud Run jobs
Static site	S3 + CloudFront / Amplify	Static Web Apps	Firebase Hosting

DO & DON'T

- ✓ **Do** Containerize even if you start on a PaaS: it keeps options open
- ✓ **Do** Compare monthly cost at expected load, not list prices
- ✗ **Don't** Choose Kubernetes because it is popular
- ✗ **Don't** Pick functions for long-running, CPU-heavy work

LOCAL TESTING

- ⌂ Docker Compose runs the same images as any platform above
- ⌂ kind / minikube for Kubernetes; Functions emulators for functions (sheet 15)
- ⌂ Load-test locally with k6 to estimate instance counts

PRODUCTION CHECKLIST

- ☐ Autoscaling limits set on every platform
- ☐ Health checks and graceful shutdown handled (SIGTERM)
- ☐ Choice documented with the expected cost at normal and peak load

Object, Block & File Storage

THREE KINDS OF STORAGE

Object storage (S3, Azure Blob Storage, Cloud Storage) keeps files as objects in buckets, reached over HTTPS, with near-unlimited size and very high durability. **Block storage** (EBS, managed disks, Persistent Disk / Hyperdisk) is a virtual disk attached to one VM. **File storage** (EFS, Azure Files, Filestore) is a shared network file system many machines mount at once.

KEY POINTS

- Object storage is the default for uploads, backups, logs, static sites and data lakes
- Durability: designed for 99.999999999% (11 nines) on object storage
- Colder tiers cost less to store and more to read
- Lifecycle rules move or delete objects by age automatically

COPY, LIST, TIER

```
# Upload and list
aws s3 cp report.pdf s3://acme-docs/2026/report.pdf
az storage blob upload --account-name acmedocs -c docs \
    -f report.pdf -n 2026/report.pdf --auth-mode login
gcloud storage cp report.pdf gs://acme-docs/2026/report.pdf

aws s3 ls s3://acme-docs/2026/
az storage blob list --account-name acmedocs -c docs \
    --prefix 2026/ --auth-mode login -o table
gcloud storage ls gs://acme-docs/2026/

# Store infrequently-read data in a colder class
aws s3 cp backup.tar s3://acme-backup/ --storage-class STANDARD_IA
az storage blob upload --account-name acmebackup -c b \
    -f backup.tar --tier Cool --auth-mode login
gcloud storage cp backup.tar gs://acme-backup/ --storage-class=NEARLINE
```

A LIFECYCLE RULE (GOOGLE CLOUD JSON)

```
{ "rule": [
  { "action": { "type": "SetStorageClass", "storageClass": "COLDLINE" },
    "condition": { "age": 90 } },
  { "action": { "type": "Delete" }, "condition": { "age": 365 } } ] }
```

CODE EXPLAINED

- → "Folders" are just key prefixes like `2026/`
- → `--auth-mode login` uses your Entra ID login instead of account keys
- → Each provider names its colder classes differently (table)
- → The lifecycle JSON moves objects to Coldline at 90 days and deletes them at a year

ACROSS THE CLOUDS

- ↔ **AWS** S3, EBS, EFS (aws-28, aws-29, aws-30)
- ↔ **Azure** Blob Storage, managed disks, Azure Files (az-28, az-29, az-30)
- ↔ **Google Cloud** Cloud Storage, Persistent Disk / Hyperdisk, Filestore (gcp-28, gcp-29, gcp-30)

RULE OF THUMB

- ☞ Object storage unless an app needs a disk or a shared POSIX mount
- ☞ Turn on versioning or soft delete for anything you cannot recreate
- ☞ Add a lifecycle rule the day you create a log or backup bucket

STORAGE CLASSES

ACCESS	AWS S3	AZURE BLOB	CLOUD STORAGE
Frequent	Standard	Hot	Standard
Monthly	Standard-IA	Cool	Nearline
Quarterly	Glacier Instant Retrieval	Cold	Coldline
Yearly / archive	Glacier Deep Archive	Archive	Archive
Automatic	Intelligent-Tiering	Lifecycle / smart tier	Autoclass

WHICH KIND

NEED	USE
Files over HTTP, any scale	Object
A VM boot or database disk	Block
Shared folder for many VMs	File
Archive for compliance	Object, archive class

DO & DON'T

- ✓ **Do** Use one bucket per purpose and environment
- ✓ **Do** Block public access at the account / organization level
- ✗ **Don't** Use object storage as a database with constant small rewrites
- ✗ **Don't** Put archive-class data where it is read often (retrieval fees add up)

LOCAL TESTING

- ⌂ LocalStack emulates S3; Azurite emulates Blob, Queue and Table storage
- ⌂ The fake-gcs-server container emulates Cloud Storage APIs
- ⌂ MinIO gives an S3-compatible server for local testing

PRODUCTION CHECKLIST

- ☐ Public access blocked; access through identities or signed URLs
- ☐ Versioning / soft delete and lifecycle rules configured
- ☐ Replication or backups for data you cannot lose

Managed Databases

managed relational (RDS, Azure SQL, Cloud SQL) · NoSQL (DynamoDB, Cosmos DB, Firestore) · caches · replicas & failover · backups & point-in-time restore · pointer to SQL & NoSQL sets

DATABASES WITHOUT THE DATABASE SERVER

Managed databases handle patching, backups, replication and failover so you only manage schemas, queries and access. **Relational** services run PostgreSQL, MySQL or SQL Server. **NoSQL** services trade joins for massive scale and predictable latency. **Caches** (Redis-compatible) cut read load. You still choose sizes, high availability, backups and network access.

KEY POINTS

- High availability = a standby in another zone with automatic failover
- Read replicas scale reads; writes still go to the primary
- Point-in-time restore rewinds to any second within the retention window
- See the SQL and NoSQL sets for query languages and data modeling

A SMALL POSTGRESQL DATABASE

```
# AWS RDS
aws rds create-db-instance --db-instance-identifier shop-db \
    --engine postgres --db-instance-class db.t4g.micro \
    --allocated-storage 20 --master-username app \
    --manage-master-user-password --multi-az --no-publicly-accessible

# Azure Database for PostgreSQL Flexible Server
az postgres flexible-server create -g rg-data -n shop-db \
    --tier GeneralPurpose --sku-name Standard_D2ds_v5 --storage-size 32 \
    --high-availability ZoneRedundant --public-access None

# Google Cloud SQL
gcloud sql instances create shop-db --database-version=POSTGRES_17 \
    --edition=ENTERPRISE --tier=db-custom-1-3840 --region=us-central1 \
    --availability-type=REGIONAL --no-assign-ip --network=vpc-app
```

CODE EXPLAINED

- Each command creates PostgreSQL with high availability across zones
- `--manage-master-user-password` keeps the password in Secrets Manager
- Public access is off in all three; reach them from the private network
- HA roughly doubles the instance cost

ACROSS THE CLOUDS

- **AWS** RDS / Aurora, DynamoDB, ElastiCache (aws-31, aws-32, aws-33)
- **Azure** Azure SQL, PostgreSQL Flexible Server, Cosmos DB, Managed Redis (az-31, az-32, az-33)
- **Google Cloud** Cloud SQL / AlloyDB, Firestore / Spanner, Memorystore (gcp-31, gcp-32, gcp-33)

RULE OF THUMB

- Default to managed PostgreSQL unless you have a reason not to
- Pick NoSQL for known access patterns at very large scale, not to avoid schemas
- Test restores, not just backups

SERVICES

TYPE	AWS	AZURE	GOOGLE CLOUD
PostgreSQL / MySQL	RDS, Aurora	Flexible Server	Cloud SQL, AlloyDB
SQL Server	RDS for SQL Server	Azure SQL	Cloud SQL for SQL Server
Key-value / document	DynamoDB	Cosmos DB	Firestore, Bigtable
Global scale	Aurora DSQL / DynamoDB global tables	Cosmos DB (multi-region writes)	Spanner
Cache	ElastiCache (Valkey)	Azure Managed Redis	Memorystore (Valkey)
Analytics	Redshift, Athena	Fabric, Synapse	BigQuery

DO & DON'T

- ✓ **Do** Keep databases on private networking only
- ✓ **Do** Use IAM / Entra authentication where supported
- ✗ **Don't** Run the database on a VM to save money without counting your time
- ✗ **Don't** Leave backups at the default retention without checking compliance

LOCAL TESTING

- Run the same engine in Docker: `docker run -e POSTGRES_PASSWORD=pw postgres:17`
- DynamoDB Local, the Cosmos DB emulator and the Firestore emulator for NoSQL
- Testcontainers starts real databases inside integration tests

PRODUCTION CHECKLIST

- ☐ Multi-zone high availability and automated backups with point-in-time restore
- ☐ Deletion protection on
- ☐ Connection pooling (RDS Proxy, PgBouncer) and slow-query monitoring

Messaging & Events

queues vs topics vs event buses · at-least-once delivery · idempotency · dead-letter queues · SQS / SNS / EventBridge · Service Bus / Event Grid · Pub/Sub / Eventarc

DECOUPLING SERVICES

Messaging lets one service hand work to another without waiting. A **queue** holds messages for one consumer group to process. A **topic** fans each message out to many subscribers. An **event bus** routes events by content to targets. Almost every cloud messaging service delivers **at least once**, so consumers must be **idempotent**: processing the same message twice must be safe.

KEY POINTS

- Messages that keep failing go to a dead-letter queue for inspection
- Visibility timeout / lock duration: how long a consumer has before redelivery
- FIFO / sessions / ordering keys give ordering, at lower throughput
- Events describe what happened; commands ask for work

SEND AND RECEIVE

```
# AWS SQS queue
aws sqs create-queue --queue-name orders
aws sqs send-message --queue-url $Q --message-body '{"id": 42}'
aws sqs receive-message --queue-url $Q --wait-time-seconds 20

# Azure Service Bus queue
az servicebus queue create -g rg-msg --namespace-name sb-shop -n orders
# (send / receive go through the SDK; the CLI manages entities)

# Google Cloud Pub/Sub topic + subscription
gcloud pubsub topics create orders
gcloud pubsub subscriptions create orders-worker --topic=orders \
    --ack-deadline=60
gcloud pubsub topics publish orders --message='{"id": 42}'
gcloud pubsub subscriptions pull orders-worker --auto-ack --limit=5
```

CODE EXPLAINED

- → SQS long polling (`--wait-time-seconds 20`) cuts empty receives
- → The Azure CLI creates Service Bus entities; apps send and receive with the SDK
- → Pub/Sub separates topics (publish) from subscriptions (receive)
- → Recording the message ID with the change makes retries harmless

IDEMPOTENT CONSUMER (PSEUDOCODE)

```
def handle(msg):
    if store.seen(msg.id):        # already processed: just ack
        return ack(msg)
    with db.transaction():
        apply_order(msg.body)
        store.mark_seen(msg.id)   # same transaction as the change
    ack(msg)
```

ACROSS THE CLOUDS

- ↔ **AWS** SQS, SNS, EventBridge (aws-23, aws-24)
- ↔ **Azure** Service Bus, Event Grid, Event Hubs (az-23, az-24)
- ↔ **Google Cloud** Pub/Sub, Eventarc, Cloud Tasks (gcp-23, gcp-24)

RULE OF THUMB

- ☞ Queue for work distribution, topic for fan-out, event bus for routing by content
- ☞ Always configure a dead-letter queue and alert on it
- ☞ Assume duplicates and out-of-order delivery unless you pay for ordering

SERVICES

PATTERN	AWS	AZURE	GOOGLE CLOUD
Queue	SQS	Service Bus queue / Storage Queue	Pub/Sub subscription / Cloud Tasks
Pub/sub topic	SNS	Service Bus topic	Pub/Sub topic
Event routing	EventBridge	Event Grid	Eventarc
Streaming	Kinesis / MSK	Event Hubs	Pub/Sub, Managed Kafka
Workflow	Step Functions	Logic Apps / Durable Functions	Workflows

DO & DON'T

- ✓ **Do** Put a message ID in every message and dedupe on it
- ✓ **Do** Keep messages small; store large payloads in object storage and send a link
- ✗ **Don't** Process a message, crash before acking, and assume it will not come back
- ✗ **Don't** Use a queue as a database to query later

LOCAL TESTING

- ⌂ LocalStack emulates SQS, SNS and EventBridge
- ⌂ The Pub/Sub emulator: `gcloud beta emulators pubsub start`
- ⌂ The Service Bus emulator runs in Docker for local development

PRODUCTION CHECKLIST

- □ Dead-letter queues with alerts on depth
- □ Alarms on queue age / backlog
- □ Consumers idempotent and scaled on queue depth

Infrastructure as Code

declarative vs imperative · state & drift · Terraform / OpenTofu providers for all three · CloudFormation & CDK · Bicep · Infrastructure Manager · plan → apply

INFRASTRUCTURE IN VERSION CONTROL

Infrastructure as code describes the resources you want in files, reviews changes in pull requests and applies them with a tool. **Declarative** tools compare the desired state with what exists and make the difference. **Terraform** (and its open-source fork **OpenTofu**) works with all three clouds; each provider also has native tools: CloudFormation and CDK, Bicep, and Infrastructure Manager (managed Terraform).

KEY POINTS

- Plan before apply: review what will be created, changed or destroyed
- State tracks what the tool manages; keep it remote and locked
- Drift: someone changed a resource by hand outside the code
- Modules / constructs package reusable patterns

ONE BUCKET IN TERRAFORM PER CLOUD

```
resource "aws_s3_bucket" "logs" {
  bucket = "acme-logs-prod"
  tags   = { owner = "platform", env = "prod" }
}

resource "azurerm_storage_account" "logs" {
  name                     = "acmelogsprod"
  resource_group_name      = "rg-logs-prod"
  location                 = "eastus2"
  account_tier             = "Standard"
  account_replication_type = "ZRS"
}

resource "google_storage_bucket" "logs" {
  name                        = "acme-logs-prod"
  location                    = "US-CENTRAL1"
  uniform_bucket_level_access = true
  labels                      = { owner = "platform", env = "prod" }
}
```

CODE EXPLAINED

- → Each resource block names a type from that cloud's provider
- → Arguments differ per cloud, but the workflow is identical
- → `plan -out` saves exactly what `apply` will do
- → Native tools: CloudFormation / CDK, Bicep, Infrastructure Manager

ACROSS THE CLOUDS

- ↔ **AWS** CloudFormation, CDK, SAM, Terraform (aws-34, aws-35, aws-36)
- ↔ **Azure** Bicep, Azure Developer CLI, Terraform (az-34, az-35, az-36)
- ↔ **Google Cloud** Terraform, Infrastructure Manager, Config Connector (gcp-34, gcp-35, gcp-36)

WORKFLOW

```
terraform init          # download providers, connect remote state
terraform plan -out=tf.plan
terraform apply tf.plan
```

RULE OF THUMB

- ☞ Terraform / OpenTofu if you use more than one cloud; native tools are fine for one
- ☞ Every change through a pull request with the plan output attached
- ☞ Never edit managed resources by hand; fix the code instead

TOOLS

TOOL	CLOUDS	LANGUAGE
Terraform / OpenTofu	All	HCL
Pulumi	All	TypeScript, Python, Go, C#
CloudFormation / SAM	AWS	YAML / JSON
AWS CDK	AWS	TypeScript, Python, Java, C#
Bicep	Azure	Bicep
Infrastructure Manager	Google Cloud	HCL (managed Terraform)

REMOTE STATE

CLOUD	BACKEND
AWS	S3 (with lock file)
Azure	Storage account container
Google Cloud	Cloud Storage bucket
Any	HCP Terraform

DO & DON'T

- ✓ **Do** Keep state in a remote, encrypted, locked backend
- ✓ **Do** Pin provider versions in `required_providers`
- ✗ **Don't** Commit `terraform.tfstate` to git (it can contain secrets)
- ✗ **Don't** Apply from a laptop to production

LOCAL TESTING

- ⌂ `terraform validate` and `terraform fmt -check` run with no cloud access
- ⌂ LocalStack works as an AWS endpoint for Terraform tests
- ⌂ `terraform test` and Terratest test modules; Checkov / Trivy scan for misconfigurations

PRODUCTION CHECKLIST

- ☐ Apply only from CI with OIDC credentials (sheet 22)
- ☐ Plans reviewed in pull requests; destroy actions highlighted
- ☐ Drift detection on a schedule

Observability

KNOWING WHAT YOUR SYSTEM IS DOING

Observability rests on three signals: **metrics** (numbers over time), **logs** (event records) and **traces** (one request's path through services). Each cloud collects platform metrics and logs automatically; your app adds its own through **OpenTelemetry**, the vendor-neutral standard that every provider now accepts. **Alerts** on user-facing symptoms and **SLOs** turn data into action.

KEY POINTS

- Platform metrics come free; custom metrics and log ingestion are billed
- Structured (JSON) logs are far easier to query
- Trace IDs in logs link a log line to its request
- Alert on symptoms (errors, latency) before causes (CPU)

QUERY RECENT ERRORS

```
# AWS CloudWatch Logs Insights
aws logs start-query --log-group-name /app/api \
    --start-time $(date -d '-1 hour' +%s) --end-time $(date +%s) \
    --query-string 'fields @timestamp, @message
      | filter level = "error" | sort @timestamp desc | limit 20'

# Azure Log Analytics (KQL)
az monitor log-analytics query -w <workspace-id> --analytics-query \
  "AppTraces | where SeverityLevel >= 3 | top 20 by TimeGenerated"

# Google Cloud Logging
gcloud logging read 'resource.type="cloud_run_revision" AND
  severity>=ERROR' --freshness=1h --limit=20
```

CODE EXPLAINED

- → Each provider has its own query language: Logs Insights, KQL, Logging query language
- → All three accept OpenTelemetry traces and metrics through exporters or collectors
- → Span attributes become searchable fields
- → The same instrumented code works on every cloud

OPENTELEMETRY IN PYTHON (ANY CLOUD)

```
from opentelemetry import trace
tracer = trace.get_tracer("checkout")

def checkout(cart):
    with tracer.start_as_current_span("checkout") as span:
        span.set_attribute("cart.items", len(cart))
        return charge(cart)
```

ACROSS THE CLOUDS

- ↔ **AWS** CloudWatch metrics, logs, X-Ray / ADOT (aws-37, aws-38)
- ↔ **Azure** Azure Monitor, Log Analytics, Application Insights (az-37, az-38)
- ↔ **Google Cloud** Cloud Monitoring, Cloud Logging, Cloud Trace (gcp-37, gcp-38)

RULE OF THUMB

- ☞ Instrument with OpenTelemetry, not a vendor SDK, to stay portable
- ☞ Set log retention on day one: default retention can be forever and costly
- ☞ Every alert should have a runbook link

SERVICES

SIGNAL	AWS	AZURE	GOOGLE CLOUD
Metrics	CloudWatch Metrics	Azure Monitor Metrics	Cloud Monitoring
Logs	CloudWatch Logs	Log Analytics	Cloud Logging
Traces / APM	X-Ray / Application Signals	Application Insights	Cloud Trace
Query language	Logs Insights	KQL	Logging query language
Managed Prometheus	Amazon Managed Prometheus	Azure Monitor managed Prometheus	Managed Service for Prometheus

DO & DON'T

- ✓ **Do** Log in JSON with a request / trace ID on every line
- ✓ **Do** Define SLOs for the few user journeys that matter
- ✗ **Don't** Log full request bodies with personal data or secrets
- ✗ **Don't** Page someone for a CPU spike that users never noticed

LOCAL TESTING

- ⌂ Run an OpenTelemetry Collector plus Jaeger or Grafana LGTM in Docker
- ⌂ `docker run -p 4317:4317 otel/opentelemetry-collector` receives local traces
- ⌂ Use the console exporter during development to print spans

PRODUCTION CHECKLIST

- ☐ Alerts on error rate and latency with runbooks
- ☐ Log retention and sampling set to control cost
- ☐ Dashboards for each service's golden signals

CI/CD to the Cloud

GitHub Actions OIDC to AWS, Azure & Google Cloud · build → push → deploy · blue/green & canary · environments & approvals · pointer to GitHub set

SHIPPING FROM A PIPELINE

A deployment pipeline builds and tests the code, pushes an artifact (usually a container image), then deploys it with infrastructure as code or a deploy command. Pipelines should log in to the cloud with **OIDC federation**, not stored keys. Gradual rollouts (**blue/green**, **canary**) let you shift traffic slowly and roll back fast.

KEY POINTS

- GitHub Actions, GitLab CI, Azure Pipelines, Cloud Build and CodePipeline all work
- Protected environments add approvals before production
- Deploy immutable artifacts: the image tested is the image shipped
- Roll back by moving traffic, not by rebuilding

GITHUB ACTIONS: BUILD AND DEPLOY TO CLOUD RUN

```yaml
on: { push: { branches: [main] } }
permissions: { contents: read, id-token: write }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production          # approval gate
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: ${{ vars.DEPLOY_SA }}
      - uses: google-github-actions/setup-gcloud@v2
      - run: |
          IMG=us-central1-docker.pkg.dev/app-prod/web/api:${{ github.sha }}
          gcloud auth configure-docker us-central1-docker.pkg.dev -q
          docker build -t $IMG . && docker push $IMG
          gcloud run deploy api --image $IMG --region us-central1
```

CODE EXPLAINED

- → `id-token: write` lets the job request an OIDC token
- → The `environment` gate can require a reviewer before deploy
- → Tagging the image with the commit SHA makes every deploy traceable
- → Only the login step changes per cloud

ACROSS THE CLOUDS

- ↔ **AWS** GitHub OIDC + CodeDeploy / CodePipeline (aws-39)
- ↔ **Azure** GitHub OIDC + Azure Pipelines, slot swaps (az-39)
- ↔ **Google Cloud** GitHub OIDC + Cloud Build / Cloud Deploy (gcp-39)

SWAP THE LOGIN STEP FOR OTHER CLOUDS

```yaml
- uses: aws-actions/configure-aws-credentials@v4
  with: { role-to-assume: ${{ vars.AWS_ROLE }}, aws-region: us-east-1 }
- uses: azure/login@v2
  with: { client-id: ${{ vars.AZ_CLIENT }}, tenant-id: ${{ vars.AZ_TENANT }},
          subscription-id: ${{ vars.AZ_SUB }} }
```

RULE OF THUMB

- ☞ OIDC everywhere; delete any cloud keys stored in CI secrets
- ☞ Same artifact through dev, staging and prod
- ☞ Automate rollback triggers on error-rate alerts

ROLLOUT STRATEGIES

STRATEGY	HOW	NATIVE SUPPORT
Rolling	Replace instances in batches	ASG / VMSS / MIG, Kubernetes
Blue/green	Two environments, switch traffic	CodeDeploy, App Service slots, Cloud Deploy
Canary	Send a small % first	Lambda aliases, Container Apps revisions, Cloud Run traffic
Feature flags	Ship dark, enable later	AppConfig, App Configuration, Firebase Remote Config

DO & DON'T

- ✓ **Do** Pin third-party actions to a version or commit SHA
- ✓ **Do** Scope the cloud role to deploy-only permissions
- ✗ **Don't** Give the pipeline Owner / Administrator access
- ✗ **Don't** Rebuild images per environment

LOCAL TESTING

- ⌂ `act` runs GitHub workflows locally (cloud login steps need the real runner)
- ⌂ actionlint catches workflow syntax errors
- ⌂ Build and run the image locally exactly as the pipeline does

PRODUCTION CHECKLIST

- ☐ OIDC trust restricted to the repo, branch and environment
- ☐ Approval gate and change record for production
- ☐ Automated rollback or one-command traffic shift

Reliability & Well-Architected

AWS Well-Architected · Azure Well-Architected · Google Cloud Architecture Framework · multi-AZ vs multi-region · RTO / RPO · backup & disaster recovery tiers

DESIGNING FOR FAILURE

Hardware, zones and even regions fail; well-built systems expect it. Each provider publishes a framework of best practices organized into pillars such as reliability, security, cost, operations, performance and sustainability. Reliability starts with two numbers: **RTO** (how long you can be down) and **RPO** (how much data you can lose). They decide between multi-zone, warm standby and multi-region active-active.

KEY POINTS

- Multi-zone covers most failures at modest cost
- Multi-region is for strict RTO / RPO or regulations
- Backups are only as good as your last tested restore
- Health checks, retries with backoff and timeouts prevent cascades

RETRIES, TIMEOUTS AND A CIRCUIT BREAKER IDEA

```python
import random, time, requests

def call_with_retry(url, attempts=4, timeout=2.0):
    for n in range(attempts):
        try:
            r = requests.get(url, timeout=timeout)   # always set a timeout
            if r.status_code < 500:
                return r
        except requests.RequestException:
            pass
        sleep = min(8, 0.2 * 2 ** n) * random.uniform(0.5, 1.5)
        time.sleep(sleep)                # exponential backoff + jitter
    raise RuntimeError(f"{url} unavailable after {attempts} tries")
```

CODE EXPLAINED

- → Timeouts stop one slow dependency from tying up every worker
- → Exponential backoff with jitter avoids retry storms
- → Only retry server errors and network failures, not 4xx
- → Each provider has a review tool or advisor that checks your setup

FRAMEWORK REVIEWS

```
aws wellarchitected create-workload --workload-name shop \
    --environment PRODUCTION --lenses wellarchitected --review-owner me \
    --aws-regions us-east-1 --description "shop"
az advisor recommendation list --category HighAvailability -o table
gcloud recommender recommendations list --project=app-prod \
    --location=global --recommender=google.iam.policy.Recommender
```

ACROSS THE CLOUDS

- ↔ **AWS** Well-Architected Framework and Tool
- ↔ **Azure** Well-Architected Framework, Azure Advisor
- ↔ **Google Cloud** Architecture Framework, Recommender

RULE OF THUMB

- ☞ Agree RTO / RPO with the business before choosing an architecture
- ☞ Multi-zone first; multi-region only with a proven need
- ☞ Run a restore drill every quarter

DR TIERS

TIER	RTO	RPO	COST
Backup & restore	Hours-days	Hours	$
Pilot light	Tens of minutes	Minutes	$$
Warm standby	Minutes	Seconds-minutes	$$$
Active-active	Near zero	Near zero	$$$$

FRAMEWORK PILLARS

PILLAR	ASKS
Reliability	Will it survive failures?
Security	Is access minimal and audited?
Cost	Is spend matched to value?
Operations	Can we deploy and observe safely?
Performance	Does it meet latency at scale?
Sustainability	Is waste minimized?

DO & DON'T

- ✓ **Do** Set timeouts on every network call
- ✓ **Do** Practice failover with game days
- ✗ **Don't** Retry instantly in a tight loop
- ✗ **Don't** Claim multi-region resilience you have never tested

LOCAL TESTING

- ⌂ Toxiproxy or `tc netem` inject latency and failures locally
- ⌂ Chaos tools: AWS Fault Injection Service, Azure Chaos Studio, Chaos Mesh / Litmus on Kubernetes
- ⌂ Restore last night's backup into a scratch environment

PRODUCTION CHECKLIST

- ☐ RTO / RPO written down and tested
- ☐ Every dependency call has timeout, retry and backoff
- ☐ Runbooks for zone and region failure

Cost Management & FinOps

KEEPING THE BILL PREDICTABLE

FinOps is the habit of making cloud spend visible, owned and optimized. The loop is the same on every provider: **tag** resources so costs map to teams, **watch** with budgets and cost explorers, **optimize** by deleting idle resources, rightsizing, and buying commitments for steady use. Most waste is forgotten resources, oversized instances, and data transfer nobody priced.

KEY POINTS

- Tags (AWS, Azure) and labels (Google Cloud) drive cost allocation reports
- Billing exports to a data store allow your own analysis
- Advisors recommend rightsizing and commitments
- Unattached disks, idle IPs, old snapshots and NAT gateways are common leaks

FIND SPEND AND WASTE

```
# AWS: last month's cost by service
aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-11-01 \
    --granularity MONTHLY --metrics UnblendedCost \
    --group-by Type=DIMENSION,Key=SERVICE
aws ec2 describe-volumes --filters Name=status,Values=available

# Azure: cost advice and unattached disks
az advisor recommendation list --category Cost -o table
az disk list --query "[?diskState=='Unattached'].[name,diskSizeGb]" -o table

# Google Cloud: export billing to BigQuery, then query it
bq query --use_legacy_sql=false '
  SELECT service.description AS svc, ROUND(SUM(cost), 2) AS usd
  FROM `billing.gcp_billing_export_v1_XXXX`
  WHERE invoice.month = "202610" GROUP BY svc ORDER BY usd DESC'
gcloud recommender recommendations list --project=app-prod \
    --location=us-central1-a \
    --recommender=google.compute.instance.MachineTypeRecommender
```

CODE EXPLAINED

- → Cost Explorer groups spend by service, tag or account
- → `status=available` EBS volumes are attached to nothing
- → Azure Advisor lists rightsizing and reservation advice
- → Google Cloud billing export to BigQuery is the standard reporting path

ACROSS THE CLOUDS

- ↔ **AWS** Cost Explorer, Budgets, Compute Optimizer (aws-02, aws-16)
- ↔ **Azure** Cost Management, Advisor, Reservations (az-02, az-16)
- ↔ **Google Cloud** Billing reports, BigQuery export, Recommender (gcp-02, gcp-16)

RULE OF THUMB

- ☞ Tag first: you cannot optimize what you cannot attribute
- ☞ Delete before you optimize: idle resources are pure waste
- ☞ Commit only to the baseline you have run for 3+ months

COST TOOLS

NEED	AWS	AZURE	GOOGLE CLOUD
Explore spend	Cost Explorer	Cost analysis	Billing reports
Budgets	AWS Budgets	Budgets	Billing budgets
Rightsizing	Compute Optimizer	Advisor	Recommender
Commitments	Savings Plans / RIs	Reservations / savings plan	Committed use discounts
Raw export	CUR / Data Exports	Cost exports	BigQuery billing export

COMMON LEAKS

LEAK	FIX
Unattached disks / IPs	Delete
Idle NAT gateways	Remove from dev
Oversized VMs	Rightsize
Logs kept forever	Set retention
Cross-region traffic	Co-locate services

DO & DON'T

- ✓ **Do** Enforce required tags / labels with policy
- ✓ **Do** Shut down dev and test environments outside working hours
- ✗ **Don't** Wait for the monthly invoice to notice a spike
- ✗ **Don't** Buy commitments before rightsizing

LOCAL TESTING

- ⌂ Infracost shows cost changes in Terraform pull requests
- ⌂ Pricing calculators estimate designs before building
- ⌂ Run cost queries against a billing export sample

PRODUCTION CHECKLIST

- ☐ Budgets with anomaly detection per team
- ☐ Tag policy enforced; untagged resources reported
- ☐ Monthly FinOps review with owners

Quick Reference — aws · az · gcloud

log in · set account / subscription / project · list resources · output & query flags · storage copy · deploy a container · logs

SIGN IN & CONTEXT

TASK	AWS	AZ	GCLOUD
Log in	aws sso login	az login	gcloud auth login
App credentials	aws sso login --profile p	az login (DefaultAzureCredential)	gcloud auth application-default login
Who am I	aws sts get-caller-identity	az account show	gcloud config list
Switch context	--profile p / AWS_PROFILE	az account set -s SUB	gcloud config set project P
Default region	aws configure set region R	az config set defaults.location=R	gcloud config set compute/region R

OUTPUT & HELP

TASK	AWS	AZ
Table output	--output table	-o table
Filter / query	--query (JMESPath)	--query (JMESPath
Help	aws s3 help	az vm -h
Browser shell	CloudShell	Cloud Shell

EVERYDAY TASKS

TASK	AWS	AZ	GCLOUD
Copy file to storage	aws s3 cp f s3://b/	az storage blob upload ...	gcloud storage cp
List VMs	aws ec2 describe-instances	az vm list -o table	gcloud compute ins
Deploy container	aws apprunner / ecs	az containerapp up	gcloud run deploy
Tail logs	aws logs tail G --follow	az monitor log-analytics query	gcloud logging tai
Kubeconfig	aws eks update-kubeconfig	az aks get-credentials	gcloud container c

IDENTITY & SECRETS

TASK	AWS	AZ
Grant a role	aws iam attach-role-policy	az role assignmer
Workload identity	aws iam create-role (trust policy)	az identity creat
Read a secret	aws secretsmanager get-secret-value	az keyvault secre
Audit log	aws cloudtrail lookup-events	az monitor activ

NETWORK & DATA

TASK	AWS	AZ
Create network	aws ec2 create-vpc	az network vnet create
Firewall rule	aws ec2 authorize-security-group-ingress	az network nsg rule cr
Create database	aws rds create-db-instance	az postgres flexible-s
Queue / topic	aws sqs create-queue	az servicebus queue cr

Sheet 26

Quick Reference — Service Map

Need → AWS → Azure → Google Cloud with sheet numbers · terminology map (account / subscription / project, role / RBAC / IAM role) · free-tier highlights

SERVICE MAP

NEED	AWS	AZURE	GOOGLE CLOUD	SHEET #
Virtual machines	EC2	Virtual Machines	Compute Engine	15
Serverless functions	Lambda	Azure Functions	Cloud Run functions	20–21
Serverless containers	ECS Fargate / App Runner	Container Apps	Cloud Run	20, 26
Kubernetes	EKS	AKS	GKE	26–27
Object storage	S3	Blob Storage	Cloud Storage	28
Relational DB	RDS / Aurora	Azure SQL / PostgreSQL	Cloud SQL / AlloyDB	31
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable	32
Queue / topic	SQS / SNS	Service Bus	Pub/Sub	24
Secrets	Secrets Manager	Key Vault	Secret Manager	08
Native IaC	CloudFormation / CDK	Bicep	Infrastructure Manager	34–35
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring	37–38
Generative AI	Bedrock	Microsoft Foundry	Agent Platform (Vertex AI)	40

TERMINOLOGY

IDEA	AWS	AZURE	GOOGLE CLOUD
Billing container	Account	Subscription	Project
Lifecycle group	Tags / stack	Resource group	Project / labels
Permission set	IAM policy	Role definition	IAM role
Workload identity	IAM role	Managed identity	Service account
Private network	VPC	VNet	VPC (global)
Firewall	Security group	NSG	Firewall rule
Metadata	Tags	Tags	Labels

FREE TIER HIGHLIGHTS (CHECK CURRENT TERMS)

PROVIDER	OFFER
AWS	Credits for new accounts plus always-free tiers (Lambda requests, DynamoDB storage)
Azure	Credit for 30 days plus 12-month and always-free services
Google Cloud	Trial credit for 90 days plus always-free e2-micro, Cloud Run and storage quotas